RAPPORT DE PROJET LO21
MINI SYSTÈME EXPERT

Binome : HENNI Bilel & FABRIZIO Elio

Janvier 2025

1 Conception et Choix d'Implémentation

1.1 Structures de Données

Pour respecter les contraintes de flexibilité et la nature dynamique d'un système expert, nous avons choisi d'utiliser exclusivement des **listes chaînées dynamiques**. Ce choix permet de gérer des règles de taille variable et une base de connaissances dont le nombre de règles n'est pas figé.

- **Type `Liste` (Générique) :** Nous avons défini une structure de maillon élémentaire (`Cellule`) contenant une chaîne de caractères (`char*`) et un pointeur vers le suivant. Ce type sert de brique de base pour tout le projet.
- **Type `Regle` :** Conformément au sujet, une règle est implémentée comme une **liste simple de propositions**.
    - *Convention de structure :* Les éléments de la liste constituent la prémisse, à l'exception du dernier élément qui est la **conclusion**.
    - Cette structure évite de créer un type complexe à deux champs et simplifie la gestion mémoire.
- **Type `BC` (Base de Connaissances) :** C'est une liste chaînée de Règles. Chaque maillon pointe vers une liste `Regle`.

1.2 Architecture Modulaire

Afin de garantir une qualité d'implémentation maximale, nous avons adopté une architecture modulaire stricte séparant l'interface (fichiers `.h`) de l'implémentation (fichiers `.c`). Cela respecte le principe d'encapsulation des Types Abstraits de Données (TAD) :

- `listes.h/c` : Gestion basique des listes de chaînes.
- `expert.h/c` : Logique métier (Règles, BC, Moteur) utilisant les primitives de listes.
- `main.c` : Programme principal et jeux d'essais.

1.3 Stratégie du Moteur d'Inférence

Nous avons implémenté un moteur à **chaînage avant avec saturation**.

- **Algorithme :** Le moteur parcourt la base de règles de manière cyclique.

- **Optimisation (Réduction des règles) :** Au lieu de re-vérifier à chaque tour si une prémisse entière est incluse dans la base de faits, nous appliquons une approche de réduction : dès qu'un fait est avéré, il est **supprimé** des prémisses des règles en attente.
- **Déclenchement :** Une règle se déclenche automatiquement lorsque sa liste ne contient plus qu'un seul élément (sa conclusion), signifiant que toutes les prémisses ont été satisfaites (éliminées).

2 Algorithmes

Conformément à la consigne, les algorithmes ci-dessous sont décrits en pseudo-code utilisant les **opérations abstraites** du type Liste (`Tete`, `Reste`, `EstVide`, `AjouterFin`, `SupprimerElement`).

2.1 Type Abstrait `Regle`

Opération : Créer une règle vide

- **Profil :** `CreerRegle` : $\emptyset \rightarrow$ `Regle`
- **Algorithme :**

```
Fonction CreerRegle() : Regle
Début
    Retourner CreerListeVide()
Fin
```

Opération : Ajouter une prémisse

- **Profil :** `AjouterPremisse` : `Regle` $\times$ `Proposition` $\rightarrow$ `Regle`
- **Algorithme :**

```
Fonction AjouterPremisse(R: Regle, P: Proposition) : Regle
Début
    // On ajoute en queue pour conserver l'ordre
    Retourner AjouterFin(R, P)
Fin
```

Opération : Tester l'appartenance (Récursif)

- **Profil :** `AppartientPremisse` : `Regle` $\times$ `Proposition` $\rightarrow$ `Booléen`
- **Algorithme :**

```
Fonction AppartientPremisse(R: Regle, P: Proposition) : Booléen
Début
    // Cas d'arrêt : Règle vide ou ne contenant que la conclusion
    Si EstVide(R) OU EstVide(Reste(R)) Alors
        Retourner Faux
    Sinon Si Tete(R) = P Alors
        Retourner Vrai
    Sinon
```

```
        Retourner AppartientPremisse(Reste(R), P)
    FinSi
Fin
```

Opération : Tester si la prémisse est vide

- **Profil :** `EstPremisseVide : Regle → Booléen`
- **Algorithme :**

```
Fonction EstPremisseVide(R: Regle) : Booléen
Début
    // La prémisse est vide si la liste ne contient QUE la conclusion
    Si EstVide(R) Alors Retourner Vrai
    Si EstVide(Reste(R)) Alors Retourner Vrai
    Sinon Retourner Faux
Fin
```

2.2 Moteur d'Inférence

Le moteur sature la Base de Faits (BF) à partir de la Base de Connaissances (BC).

- **Profil :** `MoteurInference : BC × BF → BF`
- **Algorithme :**

```
Fonction MoteurInference(BaseC: BC, BaseF: Liste) : Liste
Variables Locales : Changement (Booléen), R (Regle), F (Chaine)
Début
    Répéter
        Changement <- Faux
        Pour chaque Regle R dans BaseC faire :

            // Phase 1 : Nettoyage de la prémisse avec les faits connus
            Pour chaque Fait F dans BaseF faire :
                Si AppartientPremisse(R, F) Alors
                    // On réduit la règle en supprimant la condition vérifiée
                    R <- SupprimerPropPremisse(R, F)
                FinSi
            FinPour

            // Phase 2 : Vérification de déclenchement
            Si EstPremisseVide(R) Alors
                Conclusion <- AccederConclusion(R)
                Si NON Appartient(BaseF, Conclusion) Alors
                    // Nouveau fait déduit !
                    BaseF <- AjouterFin(BaseF, Conclusion)
                    Changement <- Vrai
                FinSi
            FinSi
        FinPour
    Jusqu'à (Changement = Faux)
```

```
    Retourner BaseF
Fin
```

3 Jeux d'Essais et Résultats

Nous avons validé le système avec deux jeux d'essais distincts pour prouver sa robustesse.

3.1 Jeu d'Essai 1 : Diagnostic Automobile (Sujet)

Configuration :

- **Règles :**
  - R1 : `reservoirVide` ET `pharesFonctionnent` ET `moteurDemarre` ⇒ `problemeBougie`
  - R2 : `moteurDemarre` ET `pharesFonctionnent` ⇒ `problemeBatterie`
  - R3 : `NONmoteurDemarre` ET `pharesFonctionnent` ⇒ `problemeStarter`
- **Faits Initiaux :** `NONmoteurDemarre`, `pharesFonctionnent`.

Résultat observé :

- Le moteur déduit correctement : `problemeStarter`.
- Les règles R1 et R2 ne se déclenchent pas (conditions non remplies).

3.2 Jeu d'Essai 2 : Classification Animale (Complémentaire)

Ce test a pour but de démontrer la capacité de **chaînage avant** du moteur (une déduction en entraîne une autre).

Configuration :

- **Règles :**
  - RA1 : `aDesPoils` ET `mangeViande` ⇒ `estCarnivore`
  - RA2 : `estCarnivore` ET `aDesRayures` ⇒ `estTigre`
- **Faits Initiaux :** `aDesPoils`, `mangeViande`, `aDesRayures`.

Résultat observé :

1. Au premier passage, le moteur valide RA1 et ajoute `estCarnivore` à la base de faits.
2. Au second passage, grâce au nouveau fait `estCarnivore`, le moteur valide RA2.
3. La conclusion finale `estTigre` est ajoutée.

3.3 Trace d'Exécution Globale

Voici la sortie console obtenue :

```
=== PROJET LO21 : SYSTEME EXPERT ===

[JEU D'ESSAI 1] : DIAGNOSTIC AUTOMOBILE
---------------------------------------
Base de faits initiale : [NONmoteurDemarre] -> [pharesFonctionnent] -> NULL
>>> Lancement du moteur d'inference <<<
   [+] Regle declenchee -> Nouveau fait : problemeStarter
>>> Saturation terminee. Aucun nouveau fait deduit. <<<
Resultat Final J1 :
[NONmoteurDemarre] -> [pharesFonctionnent] -> [problemeStarter] -> NULL


[JEU D'ESSAI 2] : CLASSIFICATION ANIMALE
----------------------------------------
Base de faits initiale :
[aDesPoils] -> [mangeViande] -> [aDesRayures] -> NULL

>>> Lancement du moteur d'inference <<<
   [+] Regle declenchee -> Nouveau fait : estCarnivore
   [+] Regle declenchee -> Nouveau fait : estTigre
>>> Saturation terminee. Aucun nouveau fait deduit. <<<
Resultat Final J2 :
[aDesPoils] -> [mangeViande] -> [aDesRayures] ->
[estCarnivore] -> [estTigre] -> NULL
```

4 Conclusion

Ce projet a permis de mettre en œuvre les Types Abstraits de Données (Liste, Règle, BC) dans un contexte d'Intelligence Artificielle symbolique. L'ajout d'un second jeu d'essai confirme que le moteur gère correctement la saturation et le chaînage des règles, validant ainsi la robustesse de l'implémentation modulaire choisie.